

P2808R0: Internal linkage in the global module

Audience: EWG, CWG

S. Davis Herring <herring@lanl.gov> (Los Alamos National Laboratory)

Michael Spencer <michael_spencer@apple.com> (Apple)

February 11, 2023

Problem

Internal linkage entities, such as functions marked `static inline`, have limitations on how they can be used in the purview of a module. It is ill-formed for them to be an “exposure”, mostly meaning cases that would need them to be referenced by TUs that import a module. This is a problem due to how common these are in C headers.

[P2003](#) describes the scope of the problem as:

- Internal linkage entities are pretty common in headers for two main reasons:
 - C’s inline semantics are different from C++, and would require a non-inline definition somewhere.
 - They provide ABI isolation as you’ll never get a different version from what the compiler saw. This means you can totally change how they work without fear of breaking ABI.

A search on github finds thousands of uses of `static inline` in C headers including projects such as Wayland, OpenSSL, Clang builtins, mono. This is also used pervasively on Apple platforms in the form of `NS_INLINE` which is defined as `static inline`. Additionally, almost all inline functions in `libc++` on Apple platforms are `static inline` for ABI isolation reasons, even member functions (via an extension). Even if both of these cases could be changed, just fixing these isn’t enough. There are still at least 10s of thousands of instances of `static inline` in the wild, and it would be unfortunate if they were not usable as header units or in the global module fragment.

`libc++` has since moved to another mechanism for achieving this goal, but using internal linkage is still a common practice.

[P2691](#) further motivates this issue with:

- Subsequent experience has shown the extent of the problem is wider than anticipated in Prague. We can report that user feedback from field experience with modules has shown that this has become a significant modules adoption blocker, as P2003 anticipated.

Solution Requirements

Any solution to this problem must:

1. Make the problematic entities referenceable from other TUs
2. Not break existing code
3. Avoid increasing UB
4. Be implementable

Solution

This paper proposes that:

1. Every TU that imports an importable header gets its own header-unit for that header.
2. The following transformations are done on the global module fragment of, and each header-unit for, a given translation unit
 1. All entities with internal linkage are given module linkage associated with the importing TU, except that they are distinct from any entity to which this transformation has not been applied (i.e., they are mangled with an unutterable tag)
 2. All functions and variables with internal linkage are made inline

Giving module linkage to a header-unit imported by a non-module TU means that a unique anonymous module is created for that TU to import from. Only those two TUs are aware of it.

This solution does raise two concerns. The first is that it is possible to violate the ODR if two separate TUs that belong to the same module bring in headers with internal linkage entities with the same name but different definitions. This is very unlikely given the size of a module, and easily worked around by splitting into multiple modules.

This also means that intentional attempts to get a single object per TU end up generating a single object per module. This is a rare usage, but it does exist.

The second concern is that during BMI generation and parsing of a global module fragment, we do not know the name of the module. This requires implementations to delay some operations until the module name is known. We have talked with implementers of MSVC, EDG, and Clang who are all not concerned with this requirement.

Another change required for this solution (and that was already incorrect) is that since we have multiple header-units for a given header-file, `[module.import]/7` needs to change to say that we import the same header-file, not the same header-unit.

This solution meets all four of the above requirements, and actually reduces the scope of UB due to ODR violations.

Alternative

As an alternative to the above, we can perform the same transformation, but instead of having one entity per module, we can have one entity per non-header-unit TU. This still requires mangling as most exposures require mangling, but is closer to the model that headers behave the same as they did in C++17. This also slightly increases the cases of cross-TU ODR violations.

Wording

Relative to N4928.

`#[basic]`

`#[basic.lookup.argdep]`

Change paragraph 4:

- [...]

If the lookup is for a dependent name ([temp.dep], [temp.dep.candidate]), the above lookup is also performed from each point in the instantiation context ([module.context]) of the lookup, additionally ignoring any declaration that appears in another translation unit, is attached to the global module, and is either discarded ([module.global.frag]) or has internal linkage.

#[basic.link]

Change paragraph 3:

- The name of an entity that belongs to a namespace scope ([basic.scope.namespace]) has internal linkage if it that is the name of
 1. a variable, variable template, function, or function template that is explicitly declared static; or
 2. a non-template variable of non-volatile const-qualified type, unless
 1. it is explicitly declared extern, or
 2. it is inline or exported, or
 3. it was previously declared and the prior declaration did not have internal linkage; or
 3. a data member of an anonymous union;

has module linkage if the declaration appears in a header unit or global module fragment and internal linkage otherwise.

[Note: An instantiated variable template that has const-qualified type can have external or module linkage, even if not declared extern. — end note]

Change paragraph 4:

- [...]

has its linkage determined as follows:

7. if the enclosing namespace has internal linkage, the name has module linkage if the declaration appears in a header unit or global module fragment and internal linkage otherwise;
8. otherwise, if the declaration of the name is attached to a named module ([module.unit]) and is not exported ([module.interface]), the name has module linkage;
9. otherwise, the name has external linkage.

Insert before paragraph 8:

- A function or variable declared in a header unit or global module fragment and whose name has module linkage is implicitly an inline function or variable ([dcl.inline]).
- The *client* of a declaration that appears in a global module fragment in a module unit of a module is that module. The client of a declaration that appears in a header unit synthesized for a client *C* ([module.import]) is
 1. the named module that contains *C*, if *C* is a module unit, and
 2. *C* otherwise.

No other declaration has a client.

Change paragraph 8:

- Two declarations of entities declare the same entity if, considering declarations of unnamed types to introduce their names for linkage purposes, if any ([dcl.typedef,dcl.enum]), they correspond ([basic.scope.scope]), have the same target scope that is not a function or template parameter scope, and either

1. they appear in the same translation unit, or
2. they both declare names with module linkage and are attached to the same named module or have the same client, or
3. they both declare names with external linkage.

[Note: There are other circumstances in which declarations declare the same entity ([dcl.link], [temp.type], [temp.spec.partial]). — *end note*]

[Example:

Source file “i.h”, not an importable header:

```
static int i;
```

Translation unit #1:

```
module;  
#include "i.h"  
module A;  
extern int j;  
int &r1 = i, &j1 = j;
```

Translation unit #2:

```
module A;  
import "i.h";  
int j;  
int &r2 = i, &j2 = j;
```

Translation unit #3:

```
module B;  
import "i.h";  
int &rb = i;
```

Translation unit #4:

```
import "i.h";  
int &r = i;
```

r1 and r2 refer to the same object, as do j1 and j2; rb and r refer to separate objects. — *end example*]

Replace paragraph 9:

- If a declaration *H* that declares a name with internal linkage precedes a declaration *D* in another translation unit *U* and would declare the same entity as *D* if it appeared in *U*, the program is ill-formed.

[Note: Such an *H* can appear only in a header unit. — *end note*]

- A declaration in a global module fragment never declares the same entity as a declaration in the purview of a named module.

Change paragraph 18:

- If a declaration that appears in one translation unit names a TU-local entity declared in another translation unit ~~that is not a header unit~~, the program is ill-formed. A declaration instantiated for a template specialization ([temp.spec]) appears at the point of instantiation of the specialization ([temp.point]).

[Drafting note: /17 previously forbid using internal-linkage declarations in global module fragments, but that linkage is changed above. — end drafting note]

#[module]

#[module.interface]

Change paragraph 3:

- If a **A**n exported declaration ~~is not within a header unit~~, it shall not declare a name with internal linkage.

#[module.import]

Change and merge paragraphs 5 and 6:

- A *module-import-declaration* that specifies a *header-name* *H* imports a synthesized *header unit*, which is a translation unit formed by applying phases 1 to 7 of translation ([lex.phases]) to the source file or header nominated by *H*, which shall not contain a *module-declaration*. The client of a *module-import-declaration* is the translation unit in which it appears. For any given header or source file ([cpp.include]), a separate header unit is synthesized once for each client.

[Note: It is therefore possible that multiple copies exist of entities with module linkage in a header unit. A definition that appears in multiple translation units cannot in general refer to such entities ([basic.def.odr]). — end note]

During the synthesis of a header unit for a client *C*, the client of a *module-import-declaration* in the header unit is considered to be *C*.

[Note: All declarations within a header unit are implicitly exported ([module.interface]), and are attached to the global module ([module.unit]). — end note]

An *importable header* is a member of an implementation-defined set of headers that includes all importable C++ library headers ([headers]). *H* shall identify an importable header. ~~Given two such *module-import-declarations*:~~

1. ~~if their *header-names* identify different headers or source files ([cpp.include]), they import distinct header units;~~
2. ~~otherwise, if they appear in the same translation unit, they import the same header unit;~~
3. ~~otherwise, it is unspecified whether they import the same header unit.~~

~~[Note: It is therefore possible that multiple copies exist of entities declared with internal linkage in an importable header. — end note]~~

[*Note: A `module-import-declaration` nominating a `header-name` is also recognized by the preprocessor, and results in macros defined at the end of phase 4 of translation of the header unit being made visible as described in [cpp.import]. Any other `module-import-declaration` does not make macros visible. — end note*]

A declaration of a name with internal linkage is permitted within a header unit despite all declarations being implicitly exported ([`module.interface`]).

[*Note: A definition that appears in multiple translation units cannot in general refer to such names ([basic.def.odr]). — end note*]

A header unit shall not contain a definition of a non-inline function or variable whose name has external linkage.

Change paragraph 7:

- When a `module-import-declaration` imports a translation unit *T*, it also imports all translation units imported by exported `module-import-declarations` in *T*; such translation units are said to be *exported* by *T*. Additionally, when a `module-import-declaration` in a module unit of some module *M* imports another module unit *U* of *M*, it also imports all translation units imported by non-exported `module-import-declarations` in the module unit purview of *U*. [*Footnote: This is consistent with the lookup rules for imported names ([basic.lookup]). — end footnote*] These rules can in turn lead to the importation of yet more translation units. If any translation unit to be imported (directly or indirectly) is a header unit, the `module-import-declaration` instead imports the header unit synthesized for its client.